Swarm Intelligence

CSC3034 Computational Intelligence

Dr Tang Tiong Yew

Outline

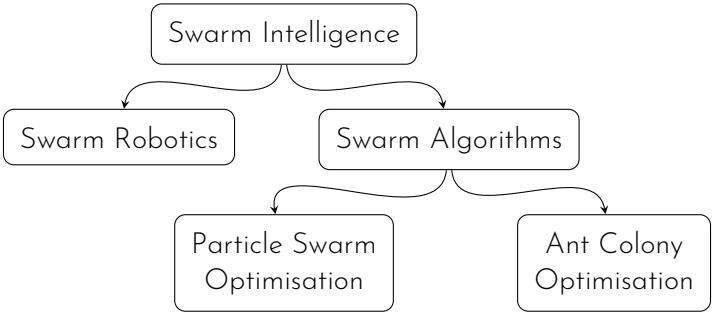
Swarm intelligence

What is swarm intelligence

Swarm intelligence is a subfield in AI focusing on the **collective** behaviour of **decentralised** and **self-organised** systems comprised of relatively **simple** agents **interacting** locally with one another and with the environment. [1]

[1] A. Iglesias, A. Gálvez, and P. Suárez, "Swarm robotics – A case study: Bat robotics", Nature-Inspired Computation and Swarm Intelligence, pp. 273–302, 2020.

Swarm intelligence



Particle swarm optimisation (PSO)

Background

- Inspired from the flocking behaviour of birds
- Each particle (individual) uses personal best and peers' best to direct itself to reach the global best position

- PSO started as a particle system (a system with multiple agents) to solve the rendering images in computer animations to create natural phenomena such as clouds, explosions, etc.
- It's later used/improved to create the flocking algorithm which simulates the flocking behaviour of birds.
- Eventually the changes made by researchers led it to be used for more generic simulation than just simulating bird models.

Basic principles

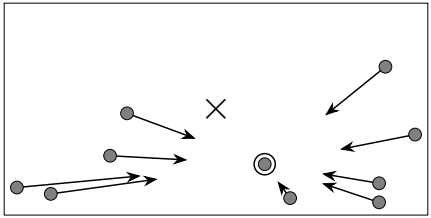
- PSO is used to find the maximum or minimum in a search space.
- a swarm of particles (individuals)
- each particle represents a potential solution
- modifications are made to the particles throughout the algorithm (as opposed to GA, where individuals are created and destroyed)
- particles are "flown" through the search space to find the best position.
- the direction towards which each particle "fly" is based on its own experience and the experience of the neighbours.

Basic principles

- parameters of a particle are its position $x_i(t)$ and velocity $v_i(t)$
- next position of a particle is calculated using

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

- the experience from the particle and its neighbours are formulated into the calculation of the velocity of the particle (the velocity update formula)

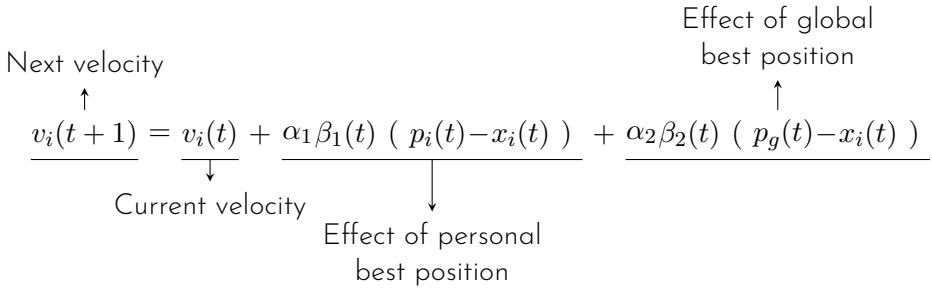


Global best PSO

- also known as gbest PSO
- neighbourhood for a particle is the entire swarm (population), i.e. all particles affect each other directly

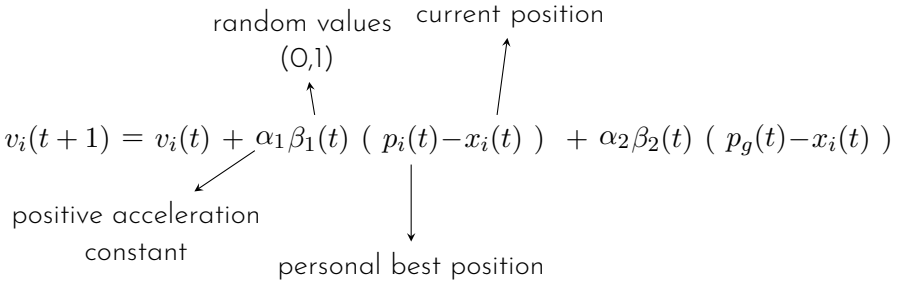
Global best PSO

The velocity of the particle is calculated as



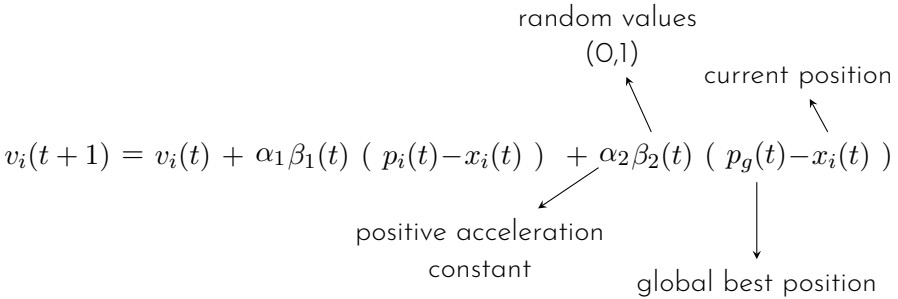
Global best PSO

The velocity of the particle is calculated as



Global best PSO

The velocity of the particle is calculated as



Global best PSO

Algorithm 1 gbest PSO

Create and initialise an n_x-dimensional swarm
repeat
  for each particle **do**
    set personal best position
    set group (global) best position
  end for
  for each particle **do**
    update velocity
    update position
  end for
until stopping condition is true

PSO implementation

1. Parameter definition
   - Define number of particles, n in the swarm
   - Determine the velocity update formula, i.e. which PSO algorithm to use
   - Define the parameters required in the velocity update formula
   - Define fitness function, i.e. determine how we decide one position is better than others
   - Identify termination condition
2. Initialise the swarm
3. Repeat until termination
   - 3.1 loop through all particles to set personal and neighbourhood best position
   - 3.2 loop through all particles to update velocity and position

Example

Problem: Find the minimum of $f(x) = x^2$ using gbest PSO.

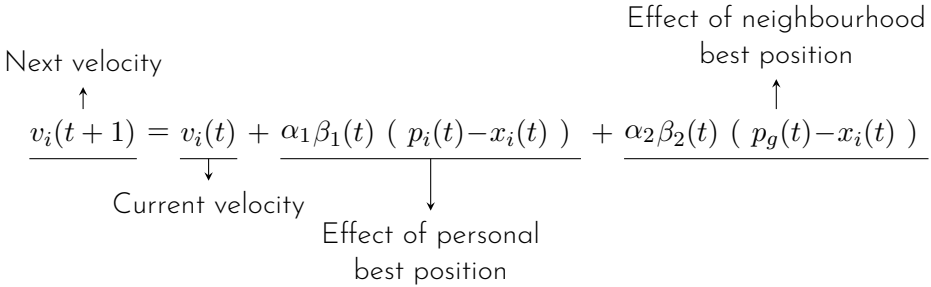
Local best PSO

- also known as lbest PSO
- a smaller neighbourhood is defined for each particle
- the velocity of the particle is calculated as

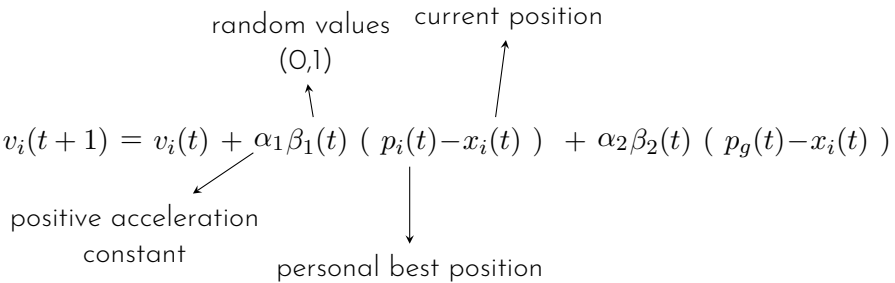
$$v_i(t+1) = v_i(t) + \alpha_1\beta_1(t)(p_i(t) - x_i(t)) + \alpha_2\beta_2(t)(p_g(t) - x_i(t))$$

the function is the same as that in gbest PSO with the exception that $p_g(t)$ is the neighbourhood best position of the particle

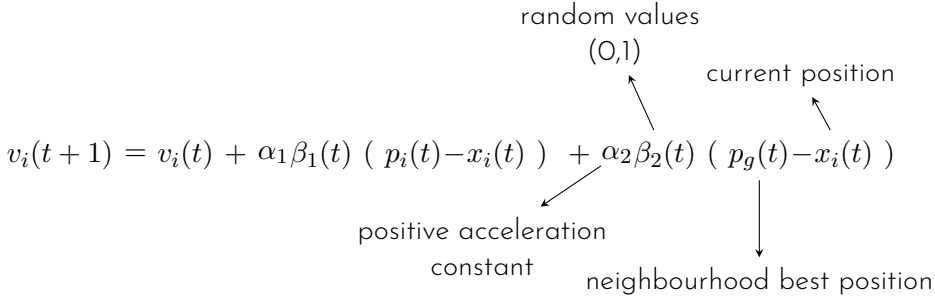
Local best PSO



Local best PSO



Local best PSO



Local best PSO

Algorithm 2 lbest PSO

Create and initialise an n_x-dimensional swarm
repeat
  for each particle **do**
    set personal best position
    set neighbourhood best position
  end for
  for each particle **do**
    update velocity
    update position
  end for
until stopping condition is true

Local best PSO

Definition of neighbourhood of a particle

- based on particle indices. If a neighbourhood is defined to have the size of n_N, the neighbourhood of the particle x_i is

  $$x_{i-n_N}(t), x_{i-n_N+1}(t), \ldots, x_{i-1}(t), x_i(t), x_{i+1}(t), \ldots, x_{i+n_N}(t)$$

- based on spatial similarity. Neighbourhood can be defined based on the (Euclidean) distance. Particles that are within a certain proximity of a particle is included in its neighbourhood.

Global best PSO vs local best PSO

- gbest PSO converges faster than lbest PSO, but less diversity
- lbest PSO has larger diversity, thus less possible to be trapped in local minima.

Termination conditions

Termination criteria

- terminate when a maximum number of iterations has been exceeded
- terminate when an acceptable solution has been found
- terminate when no improvement is observed over a number of iterations
- terminate when the particles are too close together
- terminate when the change in fitness is approximately zero

Variants

Variants of PSO are introduced to tweak different aspects of the generic PSO algorithm.

To reduce the possibility of particles flying out of the search space, **velocity clamping** is introduced. This limits the velocity of the particle v_i to $[-v_{\max}, v_{\max}]$.

Variants

In this equation,

$$v_i(t+1) = v_i(t) + \alpha_1\beta_1(t)(p_i(t) - x_i(t)) + \alpha_2\beta_2(t)(p_g(t) - x_i(t))$$

without velocity memory, $v_i(t)$,

$\rightarrow$ the swarm converges to local best solution,

with velocity memory, $v_i(t)$,

$\rightarrow$ the swarm expands to find global best solution.

Variants

Inertia weight, w was introduced to control/balance this behaviour

$$v_i(t+1) = wv_i(t) + \alpha_1\beta_1(t)(p_i(t) - x_i(t)) + \alpha_2\beta_2(t)(p_g(t) - x_i(t))$$

Alternatively, **constriction factor**, χ was introduced to control this behaviour

$$v_i(t+1) = \chi\{v_i(t) + \alpha_1\beta_1(t)(p_i(t) - x_i(t)) + \alpha_2\beta_2(t)(p_g(t) - x_i(t))\}$$

$$\text{where } \chi = \frac{2}{|2 - \alpha - \sqrt{\alpha^2 - 4\alpha}|}$$
$$\text{with } \alpha = \alpha_1 + \alpha_2 \text{ and } \alpha > 4$$

with the introduction of constriction factor, velocity clamping is no longer needed.

Cognition-only model

This variant of PSO only considers personal best and not group best. The velocity update equation becomes

$$v_i(t+1) = v_i(t) + \alpha_1 \beta_1(t)(p_i(t) - x_i(t))$$

Social-only model

This variant of PSO only considers global best and not personal best. The velocity update equation becomes

$$v_i(t+1) = v_i(t) + \alpha_2\beta_2(t)(p_g(t) - x_i(t))$$

Selfless model

This variant of PSO is basically the social model, but with the condition that, the neighbourhood best cannot be the particle itself.

Example

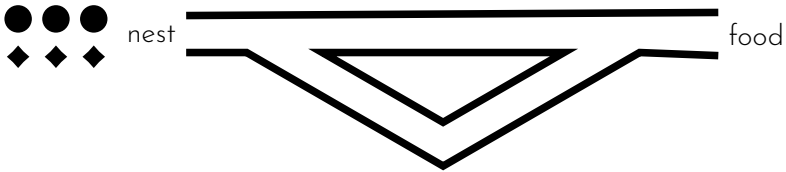
Problem: Find the maximum value of $f(x) = 15x - x^2$ assuming x can only be integers between 0 and 15

1. using gbest PSO.
2. using lbest PSO.

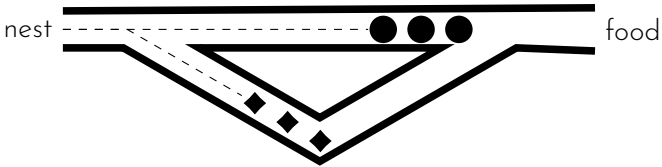
Ant colony optimisation (ACO)

Background

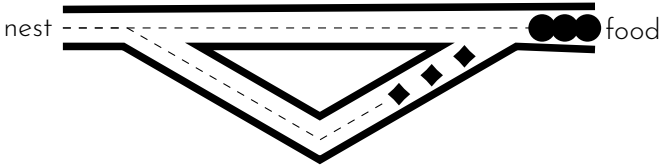
- Ants are efficient at bringing food back to their nest (foraging behaviour).
- Ants left behind a chemical pheromone trail along their path. Ants also tend to follow a path with higher pheromone concentration.



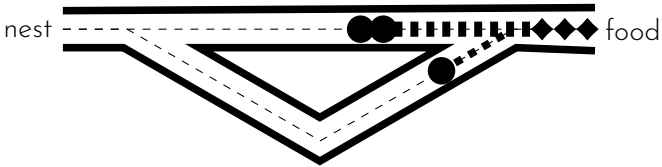
- At the initial stages of foraging, ants will take all possible routes to the food with same probability.



- As the ants on the shorter route will arrive at the food earlier, that's the only route with pheromone deposited, it is more likely that the ants will choose the shorter route to return to the nest.



- As this process repeats, the shorter route will have higher and higher pheromone concentration compare to other routes. More ants will thus choose the shorter route.

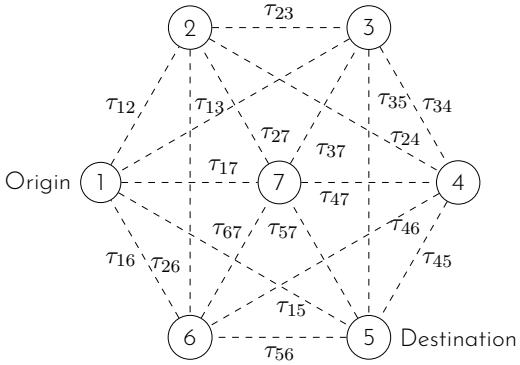


- The pheromone trail on longer route will evaporate over time.
- Eventually all ants will take the shortest route.
- This indirect communication between ants via pheromone trails is known as stigmergy.

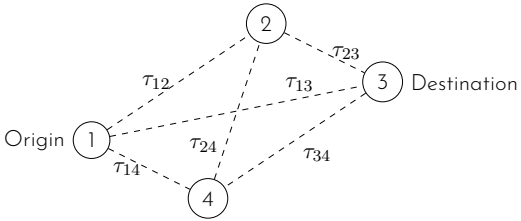
Ant colony optimisation (ACO)

- The foraging behaviour of ants inspired the introduction of the ant colony optimisation (ACO) algorithms (or some call ant colony system (ACS)).
- ACO is used to solve combinatorial optimisation problems, i.e. to identify the best (optimal) combination (ordering) from a finite set of discrete objects.
- The classic example: the **travelling salesman problem** (TSP):
  Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?

- Generally, a combinatorial optimisation problem is a problem of finding the shortest path between two nodes on a graph.



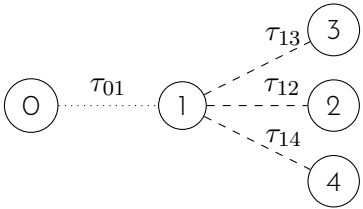
Taking a simple problem with a graph with 4 nodes.



The aim is to go from node 1 to node 3. τ_{ij} is the pheromone amount deposited on the edge (i, j).

1. Initialise the pheromone amount on each edge with small random values. Strictly speaking, edges do not have any pheromone at the beginning.
2. Start with a number of ants at the origin.
3. Identify the path of each ant from origin to destination based on the edge probability.
4. Calculate the path length of each ant.
5. Evaporate the pheromone.
6. Deposit the pheromone, i.e. calculate the increment of pheromone on each edge based on the path length and update the pheromone amount.
7. Repeat step 3 to 6 until termination condition is true.
8. The path with shortest path length is selected as the solution.

1. Initialise the pheromone amount on each edge with small random values. Strictly speaking, edges do not have any pheromone at the beginning.
2. Start with a number of ants at the origin.
3. **Identify the path of each ant from origin to destination based on the edge probability.**
4. Calculate the path length of each ant.
5. Evaporate the pheromone.
6. Deposit the pheromone, i.e. calculate the increment of pheromone on each edge based on the path length and update the pheromone amount.
7. Repeat step 3 to 6 until termination condition is true.
8. The path with shortest path length is selected as the solution.

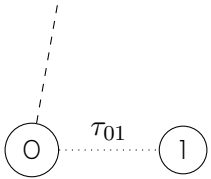


If an ant is at node 1, the probability of choosing the edge $(1, 2)$ is calculated by

$$p_{12} = \frac{\tau_{12}^{\alpha}}{\tau_{12}^{\alpha} + \tau_{13}^{\alpha} + \tau_{14}^{\alpha}}$$

where α is a positive constant to amplify the influence of pheromone.

In this case, we will consider edges $(1, 2)$, $(1, 3)$, and $(1, 4)$ only.



In cases where node 1 is not connected to any node other than node 0, we will then consider edge $(1, 0)$.

This may create loops within the constructed path. These loops are removed once the path is fully constructed, i.e. once the destination node is reached.

1. Initialise the pheromone amount on each edge with small random values. Strictly speaking, edges do not have any pheromone at the beginning.
2. Start with a number of ants at the origin.
3. Identify the path of each ant from origin to destination based on the edge probability.
4. **Calculate the path length of each ant.**
5. Evaporate the pheromone.
6. Deposit the pheromone, i.e. calculate the increment of pheromone on each edge based on the path length and update the pheromone amount.
7. Repeat step 3 to 6 until termination condition is true.
8. The path with shortest path length is selected as the solution.

The path length L^k of ant k is calculated using the function chosen based on the questions. This is equivalent to the fitness function for GA and PSO.

1. Initialise the pheromone amount on each edge with small random values. Strictly speaking, edges do not have any pheromone at the beginning.
2. Start with a number of ants at the origin.
3. Identify the path of each ant from origin to destination based on the edge probability.
4. Calculate the path length of each ant.
5. **Evaporate the pheromone.**
6. Deposit the pheromone, i.e. calculate the increment of pheromone on each edge based on the path length and update the pheromone amount.
7. Repeat step 3 to 6 until termination condition is true.
8. The path with shortest path length is selected as the solution.

The evaporation formula for an edge (i, j) is given by

$$\tau_{ij} = (1 - \rho)\tau_{ij}$$

where ρ is a constant between 0 and 1 inclusively, which specifies the rate pheromones evaporate.

1. Initialise the pheromone amount on each edge with small random values. Strictly speaking, edges do not have any pheromone at the beginning.
2. Start with a number of ants at the origin.
3. Identify the path of each ant from origin to destination based on the edge probability.
4. Calculate the path length of each ant.
5. Evaporate the pheromone.
6. **Deposit the pheromone, i.e. calculate the increment of pheromone on each edge based on the path length and update the pheromone amount.**
7. Repeat step 3 to 6 until termination condition is true.
8. The path with shortest path length is selected as the solution.

The amount of pheromone deposit by an ant k on an edge (i, j) is inversely proportional to the length of the path the ant took.

$$\Delta\tau_{ij}^{k} \propto \frac{1}{L^{k}}$$

The simplest form is $\Delta\tau_{ij}^{k} = \frac{1}{L^{k}}$.

The new pheromone concentration on the path, given that there are n_k ants in the system, is

$$\tau_{ij} = \tau_{ij} + \sum_{k=1}^{n_k} \Delta\tau_{ij}^{k}$$

1. Initialise the pheromone amount on each edge with small random values. Strictly speaking, edges do not have any pheromone at the beginning.
2. Start with a number of ants at the origin.
3. Identify the path of each ant from origin to destination based on the edge probability.
4. Calculate the path length of each ant.
5. Evaporate the pheromone.
6. Deposit the pheromone, i.e. calculate the increment of pheromone on each edge based on the path length and update the pheromone amount.
7. Repeat step 3 to 6 until **termination condition** is true.
8. The path with shortest path length is selected as the solution.

These are some common termination conditions:

- terminate when a maximum number of iterations is exceeded
- terminate when an acceptable solution has been found
- terminate when all ants or most ants follow the same path

ACO algorithm

Algorithm 3 ACO algorithm

Initialise $\tau_{ij}(0)$ to small random values or zero (0)
repeat
    Place n_k ants on the origin node
    Identify the path for each ant $\{1, \ldots, n_k\}$
    Update pheromone concentration on each edge (i, j)
until termination condition is true
Return the path with smallest path cost as solution

ACO algorithm

Algorithm 3 ACO algorithm

Initialise $\tau_{ij}(0)$ to small random values or zero (0)
repeat
  Place n_k ants on the origin node
  Identify the path for each ant $\{1, \ldots, n_k\}$
  Update pheromone concentration on each edge (i, j)
until termination condition is true
Return the path with smallest path cost as solution

---

for ant $k = 1, \ldots, n_k$ **do**
  Initiate empty path for ant k, i.e. $x^k(t) = \varnothing$
  repeat
    Select next node based on probability $p_{ij}^k(t)$ (Slide 37)
    Add edge (i, j) to path $x^k(t)$
  until destination node is reached
  Remove all loops from $x^k(t)$
  Calculate path length $L^k(t)$
end for

ACO algorithm

Algorithm 3 ACO algorithm

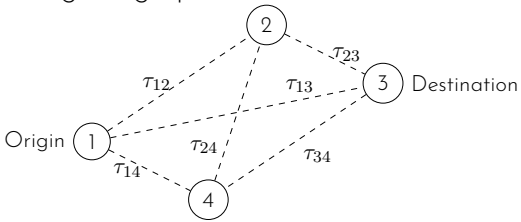
Initialise $\tau_{ij}(0)$ to small random values or zero (0)
repeat
  Place n_k ants on the origin node
  Identify the path for each ant $\{1, \ldots, n_k\}$
  Update pheromone concentration on each edge (i, j)
until termination condition is true
Return the path with smallest path cost as solution

---

for each edge (i, j) **do**
  Evaporate pheromone $\tau_{ij}(t) = (1 - \rho)\tau_{ij}(t)$
  for ant $k = 1, \ldots, n_k$ **do**
    Calculate new pheromone deposited by ant k, $\Delta\tau_{ij}^k(t) = \frac{1}{L^k(t)}$
  end for
  Update pheromone. $\tau_{ij}(t+1) = \tau_{ij}(t) + \sum_{k=1}^{n_k} \Delta\tau_{ij}^k(t)$
end for

Example

Taking the graph with 4 nodes.



Edge length is given by

Edge	Length	Edge	Length
(1,2)	3	(2,3)	10
(1,3)	20	(2,4)	1
(1,4)	5	(3,4)	3

Max-Min Ant System (MMAS)

- ACO sometimes causes prematurely stagnation for complex problems, i.e. all ants follow exactly the same path with little exploration.
- MMAS restricts the pheromone intensities to within an interval. Different strategies have been developed to achieve this restriction of pheromone intensities.

Fast Ant System

- Only one ant is used.
- Significantly reduces the computational complexity.
- A different pheromone update rule is used and no evaporation of pheromone happens.

Other swarm algorithms

- Ant colony clustering
- Bee algorithm
- Dragonfly algorithm
- Frogs algorithm
- Slime mold algorithm
- ...

Thank you

CSC3034 Current Course Update

05 Swarm Intelligence - handout

Synchronized from the current CSC3034 course site on 14 August 2026.

- Current Labs 4 and 5 cover PSO target search and ACO route finding.
- Physical examples account for robot speed limits, obstacles, and waypoint motion.
- Current runnable scripts are included in the synchronized Lab Solutions folders.

Current materials author: Dr Tang Tiong Yew

See the synchronized lab notes and runnable examples for full instructions.